

Compreendendo o *Drift* em Transformações Afins 2D

A Origem Geométrica, o Sanduíche de Pivô e a Invariância Espacial em Motores Gráficos

anicrop Engine — Documentação Teórica

23 de agosto de 2026

Índice

1	Introdução: O que é o <i>Drift</i> ?	2
2	A Álgebra das Matrizes Afins e o Sanduíche de Pivô	2
2.1	Representação por Coordenadas Homogêneas 3×3	2
2.2	O Sanduíche de Pivô (<i>Pivot Sandwich</i>)	2
3	A Geometria dos 4 Cantos e a Bounding Box (AABB)	3
3.1	O Vetor de Drift Intrínseco: $(\Delta X_{\text{drift}}, \Delta Y_{\text{drift}})$	3
4	O Caso Crítico: Crop, Rotação e Desdobramento (<i>Uncrop / Fit Content</i>)	4
4.1	Cenário 1: Camada Cropada e Rotacionada	4
4.2	Cenário 2: O Que Acontecia no <i>Fit Content</i> Sem Compensação	5
5	A Solução Algébrica Definitiva: Invariância via Matriz Inversa	5
6	Resumo dos Princípios Arquiteturais Consolidados	6

1 Introdução: O que é o *Drift*?

No contexto de motores de composição gráfica 2D (como o **anicrop**, Photoshop, Figma ou After Effects), o termo ***Drift*** refere-se a:

Definição Formal de *Drift*

O deslocamento angular e translacional espúrio que surge na caixa delimitadora (*Bounding Box*) de uma camada quando uma transformação afim (rotação, escala ou cisalhamento) é aplicada em torno de um ponto de pivô, fazendo com que os cantos transformados não comecem na origem $(0, 0)$ do espaço local.

O *drift* não é um erro de arredondamento de ponto flutuante nem um bug de código. Ele é uma consequência geométrica e trigonométrica fundamental da projeção de retângulos rotacionados em coordenadas cartesianas discretas.

2 A Álgebra das Matrizes Afins e o Sanduíche de Pivô

2.1 Representação por Coordenadas Homogêneas 3×3

Em computação gráfica 2D, qualquer transformação afim sobre um ponto $\mathbf{p} = \begin{bmatrix} x \\ y \end{bmatrix}$ é expressa na forma homogênea:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \mathbf{M} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} m_{00} & m_{01} & t_x \\ m_{10} & m_{11} & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \quad (1)$$

Onde a submatriz 2×2 superior esquerda controla rotação, escala e cisalhamento (*distorção*), e o vetor $\begin{bmatrix} t_x \\ t_y \end{bmatrix}$ controla a translação.

2.2 O Sanduíche de Pivô (*Pivot Sandwich*)

A rotação pura padrão $\mathbf{R}(\theta)$ gira em torno da origem $(0, 0)$:

$$\mathbf{R}(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad (2)$$

Para girar uma imagem de tamanho $W \times H$ em torno de um pivô arbitrário $\mathbf{P} = (p_x, p_y)$ (por exemplo, seu centro $(\frac{W}{2}, \frac{H}{2})$), o motor gráfico deve:

1. Transladar o pivô $\mathbf{P}$ para a origem: $\mathbf{T}(-\mathbf{P}) = \begin{bmatrix} 1 & 0 & -p_x \\ 0 & 1 & -p_y \\ 0 & 0 & 1 \end{bmatrix}$
2. Aplicar a rotação pura: $\mathbf{R}(\theta)$

3. Transladar a origem de volta para o pivô: $\mathbf{T}(\mathbf{P}) = \begin{bmatrix} 1 & 0 & p_x \\ 0 & 1 & p_y \\ 0 & 0 & 1 \end{bmatrix}$

A matriz resultante da rotação com pivô é o produto:

$$\mathbf{M}_{\text{pivô}} = \mathbf{T}(\mathbf{P}) \cdot \mathbf{R}(\theta) \cdot \mathbf{T}(-\mathbf{P}) \quad (3)$$

Expandindo algebricamente:

$$\mathbf{M}_{\text{pivô}} = \begin{bmatrix} \cos \theta & -\sin \theta & p_x(1 - \cos \theta) + p_y \sin \theta \\ \sin \theta & \cos \theta & p_y(1 - \cos \theta) - p_x \sin \theta \\ 0 & 0 & 1 \end{bmatrix} \quad (4)$$

A Origem Algébrica do Drift

Observe o vetor de translação introduzido automaticamente pelo sanduíche de pivô:

$$t_x(\mathbf{P}, \theta) = p_x(1 - \cos \theta) + p_y \sin \theta$$

$$t_y(\mathbf{P}, \theta) = p_y(1 - \cos \theta) - p_x \sin \theta$$

Esse vetor **muda drasticamente** conforme o tamanho da imagem ou a posição do pivô varia!

3 A Geometria dos 4 Cantos e a Bounding Box (AABB)

Considere um retângulo alinhado aos eixos com vértices:

$$C_0 = (0, 0), \quad C_1 = (W, 0), \quad C_2 = (W, H), \quad C_3 = (0, H) \quad (5)$$

Quando multiplicamos esses 4 vértices pela matriz $\mathbf{M}_{\text{pivô}}$, obtemos 4 novos pontos no espaço 2D: C'_0, C'_1, C'_2, C'_3 .

A *Axis-Aligned Bounding Box* (AABB) resultante no Canvas é definida pelos extremos:

$$X_{\min} = \min(C'_{0x}, C'_{1x}, C'_{2x}, C'_{3x}) \quad (6)$$

$$Y_{\min} = \min(C'_{0y}, C'_{1y}, C'_{2y}, C'_{3y}) \quad (7)$$

$$X_{\max} = \max(C'_{0x}, C'_{1x}, C'_{2x}, C'_{3x}) \quad (8)$$

$$Y_{\max} = \max(C'_{0y}, C'_{1y}, C'_{2y}, C'_{3y}) \quad (9)$$

3.1 O Vetor de Drift Intrínseco: $(\Delta X_{\text{drift}}, \Delta Y_{\text{drift}})$

O vetor de *drift* é exatamente a coordenada do canto superior esquerdo da Bounding Box no espaço transformado local:

$$\text{Drift}(\mathbf{M}, W, H) = (X_{\min}, Y_{\min}) \quad (10)$$

Exemplo Numérico Prático ($W = 400, H = 400, \theta = 45^\circ$):

- Pivô central: $\mathbf{P} = (200, 200)$

- $\cos(45^\circ) = \sin(45^\circ) = \frac{\sqrt{2}}{2} \approx 0.7071$
- Os cantos transformados são:

$$C'_0 = (200 - 200\sqrt{2}, 200) \approx (-82.84, 200)$$

$$C'_1 = (200, 200 - 200\sqrt{2}) \approx (200, -82.84)$$

$$C'_2 = (200 + 200\sqrt{2}, 200) \approx (482.84, 200)$$

$$C'_3 = (200, 200 + 200\sqrt{2}) \approx (200, 482.84)$$

- $X_{\min} = -82.84 \approx -83\text{px}$
- $Y_{\min} = -82.84 \approx -83\text{px}$
- Largura da Bounding Box: $482.84 - (-82.84) = 400\sqrt{2} \approx 565.68 \approx 566\text{px}$.

O *drift* dessa rotação é **Drift** = $(-83, -83)$. Se você posicionar a moldura em (X, Y) , a Bounding Box no Canvas começará em $(X - 83, Y - 83)$!

—

4 O Caso Crítico: Crop, Rotação e Desdobramento (*Uncrop / Fit Content*)

Aqui está a anatomia exata do problema que investigamos no cenário Mariachi.

4.1 Cenário 1: Camada Cropada e Rotacionada

1. Imagem base: 736×1104 .
2. Recorte (ROI): $(100, 50, 400, 400)$.
3. Rotação: $\theta = 45^\circ$. O pivô é o centro do recorte: $\mathbf{P}_{\text{crop}} = (200, 200)$.
4. Alinhamento ao Canvas: A moldura vai para $(168, 352)$.
5. O *offset* da base em relação ao recorte é:

$$\mathbf{Offset} = \mathbf{Base}_{\text{pos}} - \mathbf{Layout}_{\text{pos}} = (68, 302) - (168, 352) = (-100, -50)$$

6. A matriz física que renderiza os pixels da imagem no Canvas é:

$$\mathbf{M}_{\text{content}} = \mathbf{T}(168, 352) \cdot \mathbf{M}_{\text{pivô}}(400, 400, 45^\circ) \cdot \mathbf{T}(-100, -50) \quad (11)$$

Calculando numericamente a translação dos pixels no Canvas:

$$\mathbf{M}_{\text{content}}[0, 2] = 332.64\text{px}, \quad \mathbf{M}_{\text{content}}[1, 2] = 163.09\text{px}$$

4.2 Cenário 2: O Que Acontecia no *Fit Content* Sem Compensação

Ao chamar `fit_content`, o recorte é desfeito. O tamanho de referência passa de 400×400 para 736×1104 .

1. O novo pivô na imagem base passa a ser: $\mathbf{P}_{\text{base}} = (368, 552)$.
2. O sanduíche de pivô para a base calcula um novo vetor de translação intrínseca:

$$t_x(\mathbf{P}_{\text{base}}, 45^\circ) \approx 498.14, \quad t_y(\mathbf{P}_{\text{base}}, 45^\circ) \approx -139.19$$

3. Se a moldura for colocada em $(132, 246)$, mas a base continuar com **Offset** = $(-64, +56)$, a matriz de renderização dos pixels é corrompida:

$$\mathbf{M}_{\text{novo}}[0, 2] = 247.15\text{px} \quad (\Delta X = 332.64 - 247.15 = \mathbf{85.5\text{px}})$$

$$\mathbf{M}_{\text{novo}}[1, 2] = 157.50\text{px} \quad (\Delta Y = 163.09 - 157.50 = \mathbf{5.6\text{px}})$$

Resultado Visual: A imagem no GIMP salta 85.5px para o lado, quebrando o alinhamento visual!

5 A Solução Algébrica Definitiva: Invariância via Matriz Inversa

Para que nenhum pixel se mova no Canvas durante uma mudança de enquadramento, impomos a **Condição de Invariância Global**:

$$\mathbf{M}_{\text{content_depois}} \equiv \mathbf{M}_{\text{content_antes}} \quad (12)$$

Como a nova camada deve operar sob a matriz de distorção da nova referência $\mathbf{M}_{\text{transform}}$:

$$\mathbf{T}(x, y) \cdot \mathbf{M}_{\text{transform}} = \mathbf{M}_{\text{content_alvo}} \quad (13)$$

Multiplicando à direita pela matriz inversa $\mathbf{M}_{\text{transform}}^{-1}$:

$$\mathbf{T}(x, y) = \mathbf{M}_{\text{content_alvo}} \cdot \mathbf{M}_{\text{transform}}^{-1} \quad (14)$$

Propriedade Fundamental da Inversão Afim

Como ambas as matrizes compartilham o mesmo bloco de rotação $\mathbf{R}(45^\circ)$, o produto $\mathbf{R}(45^\circ) \cdot \mathbf{R}(-45^\circ) = \mathbf{I}$ anula a rotação completamente! O produto resultante é uma **Matriz de Translação Pura**:

$$\begin{bmatrix} 1 & 0 & x \\ 0 & 1 & y \\ 0 & 0 & 1 \end{bmatrix} \quad (15)$$

Onde x e y são exatamente as coordenadas onde a nova moldura deve ser ancorada para zerar o *drift* e o *offset* residual.

6 Resumo dos Princípios Arquiteturais Consolidados

Conceito	Responsabilidade	Regra de Ouro
<code>Transform</code>	Distorção Afim Intrínseca	Sanduíche de pivô local imutável
<code>Layout</code>	Enquadramento e Posição	Compensa o drift angular via $\mathbf{T} = \mathbf{M} \cdot \mathbf{M}_{\text{trans}}^{-1}$
<code>BaseLayer._region</code>	Geometria Canônica da Imagem	Sincronizada ao zerar o crop (<i>offset</i> = 0)
<code>Content Matrix</code>	Projeção dos Pixels no Canvas	Imutável durante operações puras de layout

Tabela 1: Separação de responsabilidades no motor geométrico do `anicrop`.